YARG Contributions — Roadmap

Canonical roadmap for both repos in this project (`yarg-song-server` and the `yarg` client fork). Status as of 2026-09-05.

Two tracks run in parallel. The **server track** is the #1 priority and is sequential — each phase depends on the one before it. The **client track** is independent and can absorb spare cycles at any time.

Phase 0 — Foundations ✅

Repos, remotes, mirrors and the format research that everything else is built on.

- [x] `fatalexception/yarg-song-server` and `fatalexception/yarg` created on Vault2 GitLab
- [x] Song-format research spec written (`docs/research/yarg-song-formats.md`)
- [x] Architecture decision recorded (`docs/ADR-001-server-architecture.md`)
- [x] Project instructions mirrored into `CLAUDE.md` so both machines pick them up
- [x] `yarg` imported from upstream server-side: 21 branches, 172 MB of repository and 180 MB of LFS objects, default branch set to `dev`
- [ ] Public push mirrors to `github.com/coffeehedake` (blocked — see Blockers)

Deliberately deferred: the `yarg` fork is *not* cloned locally. The project folder is inside the Syncthing `dev-projects` mesh, so a local clone replicates ~350 MB to r7 and Vault2 for a repo nothing touches until Phase 3. Clone it when Phase 3 starts:

```
cd "C:\dev\YARG - Open Source Contributions"
git clone https://gitlab.badassium.com/fatalexception/yarg.git yarg
cd yarg
git remote add upstream https://github.com/YARC-Official/YARG.git
git submodule update --init --recursive
git lfs pull
```

Exit criterion: both repos exist, both push to Vault2, both mirror to GitHub, and the format spec is committed.

Phase 1 — `yargsong` : the Go format library

The server is worthless until Go can read and write what YARG reads and writes. This phase is pure library work with no network surface, which makes it the easiest phase to test exhaustively.

Build order (each step is independently testable):

1. `song.ini` **reader/writer** — ~130 recognised keys, 8 scalar types. Must be lenient the way `YARGIniReader.cs` is lenient: BOM handling, non-UTF-8 charts (Latin-1 and Shift-JIS exist in the wild), duplicate keys, `[song]` / `[Song]` variance, trailing comments. Do **not** use a strict INI library.
2. `.sng` **reader** — implemented as an `fs.FS` so the folder scanner and the `.sng` scanner share exactly one code path. Fixed-offset header, little-endian, 256-byte XOR table.
3. **Scanner** — walks a folder or `.sng`, applies YARG's chart-file priority order (`notes.mid` → `notes.midi` → `notes.chart` → `notes.txt`), discovers stems/art/background, and emits the v1 catalog schema.
4. **Identity** — `SHA1(chart file bytes)`, matching YARG's `HashWrapper` exactly, so client and server agree on "do I already have this". Plus a `package_hash` of our own for same-chart-different-audio cases. Model hash → *many* entries; duplicates are normal.
5. `.sng` **writer** — validated two ways: round-trip through the reference `SngCli`, and by confirming a real YARG install scans the output and shows correct metadata.
6. **MIDI preparsers** — derive which instruments and difficulties actually exist. Only note-range walking is needed, not full chart semantics. This step is deliberately last; everything before it works with `parts_derived: false`.

Exit criterion: a CLI that scans a real song library, emits a catalog, repacks to `.sng`, and YARG scans the repacked output with identical metadata and an identical hash.

Explicitly out of scope, permanently: CON/mogg decryption, `songcache.bin` generation, `.milo_xbox` / `.png_xbox` decoding, `.yarground` inspection, full MIDI chart semantics.

Phase 2 — The server, and a sync client that needs no client changes

Two deliverables. The sync client is what makes the server *useful* before any client fork work exists, and it is the proof that the server is correct.

2a — `yarg-song-server`

- HTTP API: browse/search over the 12 attributes YARG itself sorts by, `GET /song/{hash}.sng`, and a bulk `POST /have` that takes a list of hashes and answers what the client is missing.

- Ingest: loose folders, `.sng`, and `.zip` / `.7z` of a loose folder. Refuse RB packages with a clear message.
- Config file + sane defaults; no database server required for v1 (embedded store).
- Docker image, multi-arch `linux/amd64` + `linux/arm64` (Pi), plus native macOS and Windows binaries from the same source. CI builds all of them.

2b — sync client

A small companion binary that pulls the server's library into an ordinary local songs folder. Unmodified YARG then just sees files. This ships real value in days, with zero risk to the client, and exercises the whole server end-to-end.

Exit criterion: a Pi on the LAN serves a shared library; two machines running stock YARG both see the same songs without either one being modified.

Phase 3 — Native remote song source in the client fork

Only now does the `yarg/` fork get touched. This is the upstream-facing work.

- Add an HTTP song source alongside local folders — browse, stream, cache.
- Touches YARG's song cache, so it must be designed with upstream in mind rather than bolted on.
- Upstream posture: `CONTRIBUTING.md` says nothing about networking or remote sources in any of its six tiers. That is an absence, not an endorsement — **open a discussion with YARC before building the PR**, and frame it as a content source, not a new game mode.
- PRs target `dev`. Never `master`.

Exit criterion: the fork can browse and play from a server without a sync step, and a discussion thread exists upstream.

Phase 4 — Modular features and the server config menu

The point at which the server stops being one feature and becomes a platform.

- Feature registry: each capability is opt-in and independently enable-able.
- Config UI in the server app, so a user turns on only what they want.
- Candidate modules: multi-user libraries and permissions, playlists/setlists shared across clients, scores and leaderboards, library health reporting.

Phase 5 — LLM chart generation

The long-term goal, and the phase most likely to move. It depends on every phase above working.

- Upload any song; auto-generate instrument and vocal parts across all difficulty levels.
- Produce a complete, working, **editable** package — generated charts are a starting point, not a final answer.
- **Distribution constraint, non-negotiable:** the chart/vocal tracks must be packageable and distributable *separately from the audio*, so charts can be shared without the music.
- Runs against the existing GPU arbitration on the home estate rather than a new stack.

Client track (independent of the server line)

Can be picked up at any time; does not block the server.

- **Controller compatibility** — official Rock Band and Guitar Hero instruments first. Upstream handles this through PlasticBand-Unity, so fixes may belong there rather than in YARG.
- **Graphics** — general rendering and visual improvements.
- Both are ordinary upstream contributions: fork, branch off `dev`, PR to `dev`.

Blockers

Blocker	Impact	Needed
The `Coffeehedake` GitHub PAT in 1Password (`MCP` vault, item `GitHub PAT`) returns **401 Bad credentials**	Cannot fork upstream to `coffeehedake`, cannot configure either GitHub push mirror	Jay to rotate the token and update that 1Password item

Sequencing note

Phases 1 and 2 are the whole of the "#1 priority". Nothing in phases 3–5 should start before a stock YARG client is reading songs from a self-hosted server, because every later decision depends on what that turns out to actually require.
